Autonomous Quadcopter

Project requirements document — v3 — 12-week build

Custom PCB (KiCad + JLCPCB PCBA) around STM32F411. Flight controller firmware in C + FreeRTOS. pymavlink bridging the companion computer to the FC over UART. Raspberry Pi 5 running ROS 2 and the autonomy stack. Black Pill is the zero-code-change fallback if PCB spin 1 has an error.

PARTS LIST & ESTIMATED COST

Bill of materials

Custom PCB (KiCad → JLCPCB PCBA)	
PCB fabrication, 5 boards	**$20**
PCBA — STM32F411, ICM-42688, BMP388	**$45**
Through-hole connectors kit	**$8**
ST-Link V2 SWD programmer	**$10**
Black Pill STM32F411 (fallback)	**$6**

Airframe & propulsion	
Filament — 3D printed frame	**$14**
4× 2205 2300KV brushless motors	**$30**
4-in-1 30A ESC (DSHOT capable)	**$36**
5" props x2 sets	**$14**
M3 hardware / screws	**$8**

Power	
4S 1500mAh LiPo	**$25**
UBEC 5V 3A (companion power)	**$12**
B6AC LiPo charger	**$42**
XT30 connectors + wiring	**$8**

Companion computer	
Raspberry Pi 5 4GB	**$60**
RPi camera module 3 wide	**$35**
64GB microSD (fast)	**$12**
RPi 5 active cooler	**$8**

Navigation & sensors	
M8N GPS + compass module	**$26**
PMW3901 optical flow*	**$35**

RC system	
RadioMaster Zorro TX**	**$100**
ELRS receiver	**$20**

Misc	
Silicone wire (12AWG, 22AWG)	**$15**
Heat shrink, zip ties, velcro	**$10**
Printed ArUco markers	**$3**

Estimated total	**~$567**

** Optical flow purchased only if proceeding to layer 2. ** Deduct $100 if RC transmitter already owned (~$467 total).*

CUSTOM PCB DESIGN

Hardware design guide

Design in KiCad referencing the Black Pill schematic and open-source Betaflight FC layouts. Submit to JLCPCB for fabrication and SMD assembly — only through-hole connectors need hand-soldering on arrival. Order in week 1 to receive by week 3.

Component	Part	Interface	Notes
MCU	STM32F411CEU6	—	LQFP48, 100MHz, FPU, JLCPCB-stocked
IMU	ICM-42688-P	SPI	Better vibration rejection than MPU-6000
Barometer	BMP388	I2C	Lower noise floor than BMP280
Regulator	AMS1117-3.3	—	3.3V LDO for MCU + IMU supply

Pin & peripheral requirements

– 4x DSHOT outputs — DMA-mapped TIM1/TIM3 channels

– UART1 — M8N GPS at 9600 baud

– UART2 — pymavlink UART to RPi at 115200 baud

– I2C1 — BMP388 barometer

– SPI1 — ICM-42688 at 8MHz+

– SWD header — 4-pin 2.54mm for ST-Link V2

– USB-C — power input and DFU flashing

– ADC — voltage divider on battery input

– 5V pad — UBEC input for ESC signal lines

– Vibration isolation mount holes — M3, matched to printed frame

Layout rules

– IMU ground plane — continuous pour, no trace cuts beneath

– 100nF + 10uF decoupling on ICM-42688 VDD within 0.5mm of pins

– Thermal relief on ESC pads — spoke pattern not solid fill

– Motor wires exit away from signal traces

– SWD and UART pads accessible without removing board

– Verify all footprints against manufacturer drawings

– Order 5 boards minimum — first-spin rework is expected

– Place baro away from motor-induced airflow

Black Pill fallback — end of week 3

If PCB spin 1 does not come up, drop in the Black Pill STM32F411 dev board. Identical chip, same firmware, minor pin remaps only. Order spin 2 immediately and continue development in parallel. Custom PCB remains a deliverable when spin 2 arrives.

SOFTWARE ARCHITECTURE

Stack overview

Three distinct software layers, each with a clear owner and interface boundary. The FC firmware and companion stack communicate exclusively via MAVLink over UART — neither layer knows or cares how the

other is implemented internally.

Layer	Runs on	Language / framework	Responsibility
Flight controller firmware	STM32F411	C + FreeRTOS	IMU read, attitude estimation, PID loops, DSHOT ESC output, MAVLink UART
MAVLink bridge	Raspberry Pi 5	Python — pymavlink	UART framing, CRC validation, message routing between FC and ROS 2 nodes
Autonomy stack	Raspberry Pi 5	Python — ROS 2 Jazzy	State machines, waypoint nav, velocity setpoint generation, CV pipeline
Computer vision	Raspberry Pi 5	Python — OpenCV	ArUco detection, 6DoF pose estimation, YOLOv8n inference (layer 3)

FreeRTOS task architecture

Task	Rate	Peripherals	Outputs
IMU task	1 kHz	SPI1 — ICM-42688	Raw gyro + accel to shared queue
Estimator task	1 kHz	Queue consumer	Roll/pitch/yaw angles via Mahony filter
Control task	400 Hz	Attitude angles, setpoints	Motor commands via cascaded PID
DSHOT task	1 kHz	TIM1/TIM3 DMA	ESC throttle packets to all 4 motors
Comms task	Event-driven	UART2	Parse incoming MAVLink, emit telemetry
Baro task	50 Hz	I2C1 — BMP388	Altitude estimate for hold mode

pymavlink — companion to FC bridge

pymavlink runs inside the ROS 2 Python process and owns the UART serial connection to the flight controller. It handles MAVLink v2 packet framing, per-message CRC validation, and deserialization into Python objects. The autonomy nodes call it to send velocity setpoints (SET_POSITION_TARGET_LOCAL_NED) and arm/disarm commands (COMMAND_LONG), and read telemetry (ATTITUDE, LOCAL_POSITION_NED, BATTERY_STATUS). All MAVLink message definitions for the custom FC dialect are declared once and shared across all ROS 2 nodes via a thin wrapper module.

APPLICATION LAYERS

Progressive autonomy stack

1 Autonomous precision landing

Baseline

Downward RPi camera detects an ArUco marker and estimates 6DoF pose via OpenCV. A velocity-controlled descent state machine on the companion computer sends SET_POSITION_TARGET_LOCAL_NED velocity setpoints via pymavlink over UART to the FC. Drone detects, centers, and lands within 10cm of marker center. Fully decoupled from firmware — testable against any MAVLink FC.

OpenCV ArUco · pose estimation · pymavlink UART · ROS 2 · velocity setpoints · downward camera

2 GPS-denied indoor navigation

Mid

PMW3901 optical flow fused with BMP388 barometer replaces GPS for indoor position hold and waypoint following. State estimator on RPi 5 fuses flow and baro into a position estimate, feeds corrections to the FC via pymavlink. More tightly coupled to firmware — begin only after layer 1 is stable.

PMW3901 · optical flow · BMP388 fusion · state estimation · GPS-denied · waypoint nav

3 Object tracking and follow

Stretch

YOLOv8n INT8 quantized runs onboard at 8-12 FPS on RPi 5. Yaw and forward velocity setpoints via pymavlink steer the drone to maintain offset distance on a moving target. No gimbal — yaw-body tracking only. Attempt only if layers 1 and 2 are complete with time remaining.

YOLOv8n · INT8 quantization · forward camera · yaw control · target follow · onboard inference

---

TIMELINE & PARALLEL WORKSTREAMS

12-week schedule

● Firmware / hardware ● Application milestone ● Checkpoint

Week	CS — PCB + firmware + app layer	AE — frame + hardware integration
1–2	KiCad schematic from Black Pill reference. Route PCB: STM32F411, ICM-42688, BMP388, LDO, ESC pads, UART headers, SWD. Generate gerbers + BOM + CPL. Order to JLCPCB PCBA. Set up STM32 toolchain (CLion + OpenOCD + ST-Link) and begin FreeRTOS task skeleton on Black Pill. ● **PCB ordered to JLCPCB**	3D print frame. Motor and ESC installation. XT30 power wiring. Solder motor leads. Prop balance. CG verification with full payload estimate.

Week	CS — PCB + firmware + app layer	AE — frame + hardware integration
2–3	FreeRTOS task skeleton on Black Pill. SPI IMU read verified. UART loopback test. pymavlink installed on RPi 5, HEARTBEAT send/receive confirmed over serial.	Frame assembly complete. RPi 5 mount and camera fixture. ESC continuity check. ROS 2 Jazzy install on RPi.
Wk 3	*PCB arrives. Power-on: 3.3V rail, SWD handshake, ICM-42688 SPI ack, BMP388 I2C ack. If any peripheral dead, drop in Black Pill and order spin 2 immediately.*	
4–5	Mahony filter on ICM-42688. Attitude estimation verified on tilt jig. Rate PID task. DSHOT output via DMA timer. All four ESCs responding to commands. ● **IMU stable, ESCs responding**	ArUco detection pipeline on RPi: camera calibration, marker detection, 6DoF pose estimation in ROS 2 node. Tested statically. pymavlink wrapper module written.
6–7	Attitude PID outer loop. BMP388 altitude hold. MAVLink telemetry on UART2. pymavlink on RPi confirmed receiving attitude packets. First tethered hover outdoors. ● **Stable hover on custom PCB**	MAVLink offboard velocity setpoints from RPi to FC via pymavlink. Closed-loop position hold verified on bench.
Wk 7	*Checkpoint — if hover not stable, spend week 8 on PID tuning before app layer work begins.*	
8–9	PID tuning from flight logs. 10 consecutive stable hovers as pass criterion. Layer 1 descent state machine integrated with live flight system. ● **Precision landing demo complete**	Layer 1 outdoor flight testing. Tune descent velocity controller. Target: land within 10cm of marker center, 3 of 5 attempts.
10–12	If layer 1 solid: PMW3901 optical flow integration for layer 2. Otherwise: harden layer 1, record demo video, portfolio writeup and KiCad files cleaned for GitHub. ● **Layer 2 begun or layer 1 polished**	System docs, wiring diagram. PMW3901 physical integration if proceeding. Flight log analysis.

FAILURE MODES & ANALYSIS

Risk assessment

PCB spin 1 has a layout or component error

High

Wrong footprint, missing trace, or PCBA placement error means board doesn't come up. Most likely: LDO wrong orientation, IMU footprint mismatch.

Mitigation: Drop in Black Pill STM32F411 same day. Order spin 2. Firmware continues unchanged — same chip, minor pin remaps. Custom PCB remains a deliverable when spin 2 arrives.

IMU noise from motor vibration

High

Brushless motors induce vibration into frame. Corrupted ICM-42688 readings manifest as attitude oscillation or uncommanded roll/pitch divergence.

Mitigation: Vibration dampening standoffs under PCB mount. Software low-pass filter on gyro. Balance props before every session. Log raw IMU and inspect frequency spectrum.

DSHOT DMA misconfiguration on STM32F411

High

DSHOT requires DMA-mapped timer channels. Incorrect TIM/DMA pairing causes sporadic motor cutouts or wrong RPM. Failure mid-flight causes crash.

Mitigation: Bench-test all four ESC channels on tilt jig before first flight. Log commanded vs actual RPM. Fallback: standard PWM 50Hz if DSHOT proves unreliable.

pymavlink UART timing causing dropped packets

Med

Python serial reads are blocking. If the ROS 2 event loop stalls, incoming MAVLink packets queue up in the kernel buffer and arrive in bursts, causing stale telemetry.

Mitigation: Run pymavlink in a dedicated Python thread with a queue feeding ROS 2 nodes. Set UART hardware flow control if available. Log packet timestamps and inspect dropout patterns.

RPi 5 brownout mid-flight

Med

RPi draws up to 5W under CV inference. Shared UBEC under voltage spike causes brownout and drops MAVLink connection.

Mitigation: Dedicated UBEC for RPi. FC-side failsafe: if MAVLink heartbeat absent for 1s, fall back to RC-controlled hover. Watchdog process on RPi restarts nodes on crash.

ArUco detection unreliable in outdoor light

Med

Direct sunlight causes overexposure. Detection drops below usable rate. Descent controller receives stale or absent pose estimates.

Mitigation: Disable auto-exposure, set manual shutter. Test in overcast conditions first. Abort descent if pose dropout exceeds 500ms.

5" frame insufficient payload capacity

Med

RPi 5 + camera + GPS + custom PCB + battery may approach thrust-to-weight limits, giving under 8 min flight time.

Mitigation: Calculate all-up weight before first print. If over 650g, switch to 7" frame with 2306 motors. PCB does not change.

Optical flow unusable on smooth floors (layer 2)

Low

PMW3901 requires textured surfaces at low altitude. Smooth floors cause flow failure and velocity drift.

Mitigation: Test flow sensor statically on target surfaces before integration. If floors are unusable, scope layer 2 as GPS-aided outdoor waypoint navigation.

Hard abort condition — end of week 7

If the drone is not hovering stably under custom firmware by end of week 7 — whether on custom PCB or Black Pill fallback — flash PX4 onto a Pixhawk. The pymavlink bridge, ROS 2 autonomy stack, and all three application layers are unchanged. The custom PCB and firmware work are documented separately as hardware engineering artifacts.

RUST OPTIONS

Optional extensions — engineering justification

The primary stack uses C + FreeRTOS for firmware and pymavlink for the MAVLink bridge. Both decisions are correct for this timeline. The two options below are where Rust has genuine engineering justification — not as resume padding but as the better tool at that specific layer. Either can be added without changing the rest of the stack.

R1 Embassy-rs — async Rust flight controller firmware

What it does

Replaces C + FreeRTOS on the STM32F411 entirely. Embassy is an async Rust embedded framework — no_std, no heap, no OS. Each FreeRTOS task becomes an async Rust function that yields at .await points. The IMU SPI driver, DMA DSHOT output, Mahony filter, PID loops, and MAVLink UART are all written in Rust. The SPI1 peripheral is an owned value passed into the IMU driver — the compiler guarantees nothing else touches it. DMA transfers are futures: the buffer is borrowed for the duration of the transfer and cannot be accessed until it completes.

Why Rust specifically

Hardware ownership is enforced at compile time. In FreeRTOS C, two tasks accessing the SPI bus simultaneously is a runtime bug — a data race you find by crashing a drone. In Embassy, the SPI handle is an owned value. Lending it to a second task simultaneously is a compile error. Similarly, DMA buffer aliasing — modifying a buffer while DMA is reading it — is a known C footgun that Embassy's lifetime system makes impossible. Concurrency bugs in the control loop become type errors, not runtime crashes.

Tradeoff vs. primary approach

C wins on ecosystem maturity for STM32F4 specifically. Every HAL example, every DMA timer configuration guide, every StackOverflow answer for DSHOT on STM32 is in C. Embassy STM32F4 support is real but documentation is thinner and debug tooling (probe-rs, defmt) has more sharp edges than OpenOCD + GDB. On a 12-week timeline, the learning cost of async Rust + embedded simultaneously is real. Recommended as a v2 firmware rewrite after the C version flies.

embassy-rs · async embedded · no_std · STM32F411 · DMA futures · owned peripherals

R2 Rust MAVLink bridge — typed protocol daemon on RPi

What it does

Replaces pymavlink with a standalone Rust binary that owns the UART serial connection to the FC. It reads a continuous byte stream, frames MAVLink v2 packets (sync byte, length, CRC), validates each packet's checksum using the per-message CRC_EXTRA seed, and deserializes payloads into typed Rust enums. A HEARTBEAT is a Heartbeat struct. A SET_POSITION_TARGET_LOCAL_NED is a distinct type. The daemon exposes parsed telemetry to ROS 2 Python nodes via a Unix domain socket and accepts outgoing commands on the same channel.

Why Rust specifically

MAVLink has 230+ message types. In pymavlink you parse to dicts — nothing prevents treating a HEARTBEAT as a position target message. In Rust the parsed result is an enum: pattern matching is exhaustive, and handling the wrong variant is a compile error. Additionally, the bridge is a long-running daemon that cannot be restarted mid-flight. Python has GC pauses that grow as the process accumulates message history over a 10-minute flight. Rust has no GC — UART forwarding latency is bounded and stable from arm to disarm.

Tradeoff vs. primary approach

pymavlink is simpler to implement and debug, is the community standard, and at 50Hz command rates is not a performance bottleneck on RPi 5. The Rust bridge adds a week of implementation for engineering properties that pymavlink mostly gets right in practice. The Rust option is strictly better as an engineering artifact and as a signal, but pymavlink is the correct default if timeline is tight. Unlike R1, this can be added mid-project without disrupting firmware development.

Rust on Linux · MAVLink v2 · binary protocol · nom · typed enum dispatch · Unix socket